```
!pip install numpy
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
```

```
!pip install matplotlib
```

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17
```

```
import numpy as np
import matplotlib.pyplot as plt

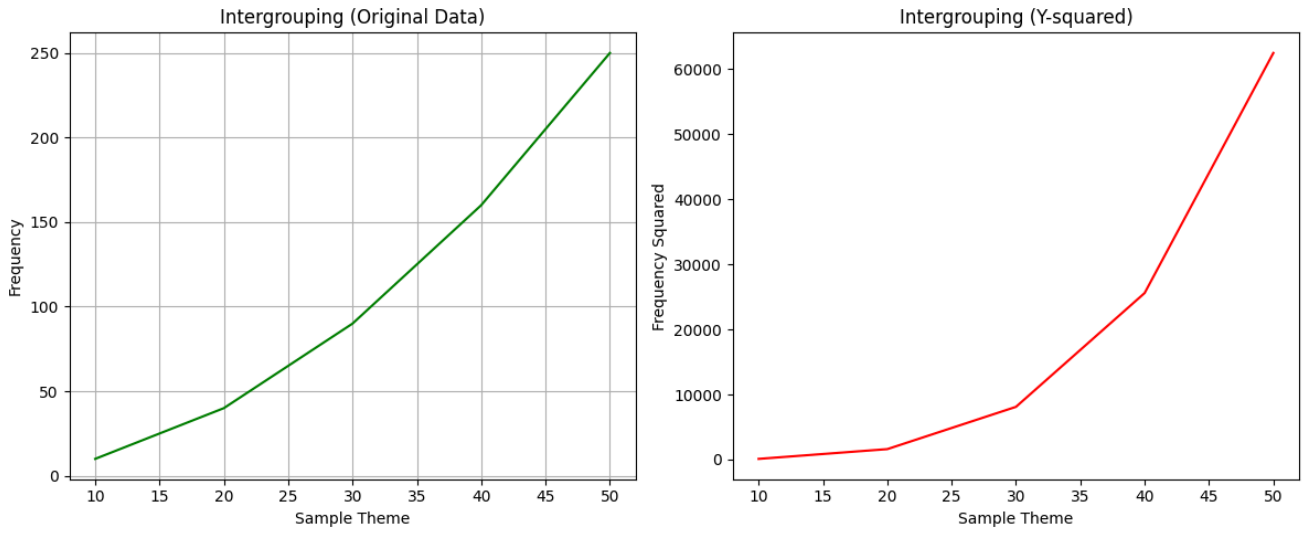
x = np.array([10, 20, 30, 40, 50])
y = np.array([10, 40, 90, 160, 250])

# Create a figure with two subplots
plt.figure(figsize=(12, 5))

# First subplot: Original data
plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
plt.plot(x, y, linestyle='-', color='green')
plt.xlabel("Sample Theme")
plt.ylabel("Frequency")
plt.title("Intergrouping (Original Data)")
plt.grid(True)

# Second subplot: Modified data (e.g., y squared)
plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
y_squared = y**2
plt.plot(x, y_squared, linestyle='-', color='red')
plt.xlabel("Sample Theme")
plt.ylabel("Frequency Squared")
plt.title("Intergrouping (Y-squared)")

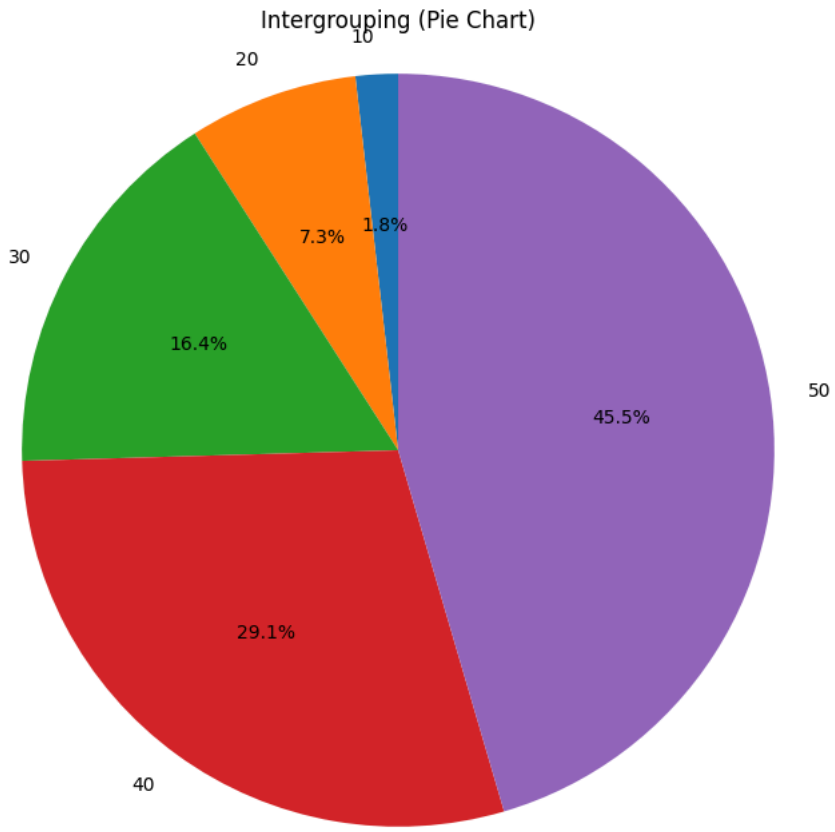

plt.tight_layout() # Adjust layout to prevent overlapping titles/labels
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([10, 20, 30, 40, 50])
y = np.array([10, 40, 90, 160, 250])

plt.figure(figsize=(8, 8))
plt.pie(y, labels=x, autopct='%1.1f%%', startangle=90)
plt.title("Intergrouping (Pie Chart)")
plt.axis('equal')  # Equal aspect ratio ensures that pie is drawn as a circle.
plt.show()
```




```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

# Load dataset
data = load_iris()
X, y = data.data, data.target

# Train model
model = DecisionTreeClassifier()
model.fit(X, y)

# Predict first sample
prediction = model.predict([X[0]])

# Display result clearly
print("Predicted Class:", prediction)
print("Predicted Flower Name:", data.target_names[prediction][0])
```

```
Predicted Class: [0]
Predicted Flower Name: setosa
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
```

```
    Dense(10, activation='relu', input_shape=(5,)),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')

print("Model created successfully")
```

```
Model created successfully
```

```
from scipy import linalg
import numpy as np

A = np.array([[2, 1], [3, 4]])
B = np.array([8, 18])

X = linalg.solve(A, B)
print(X)
```

```
[2.8 2.4]
```

```
import numpy as np
import matplotlib.pyplot as plt

# Generate x values
x_values = np.linspace(0, 2 * np.pi, 100)

# Generate sine wave signal
sine_wave = np.sin(x_values)

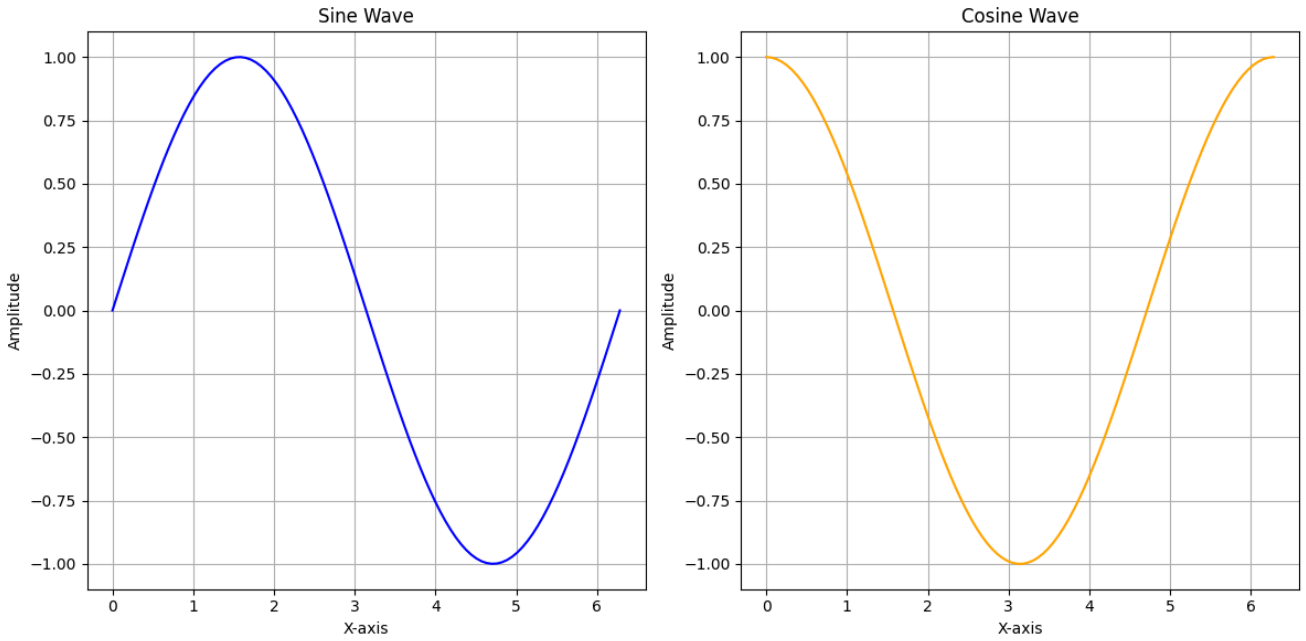
# Generate cosine wave signal
cos_wave = np.cos(x_values)

# Create a figure with two subplots
plt.figure(figsize=(12, 6))

# First subplot: Sine Wave
plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
plt.plot(x_values, sine_wave, color='blue')
plt.title("Sine Wave")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Second subplot: Cosine Wave
plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
plt.plot(x_values, cos_wave, color='orange')
plt.title("Cosine Wave")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

plt.tight_layout() # Adjust layout to prevent overlapping titles/labels
plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt

# Generate x values
x_values = np.linspace(0, 2 * np.pi, 100)

# Generate sine wave signal for the first subplot
sine_wave_1 = np.sin(x_values)

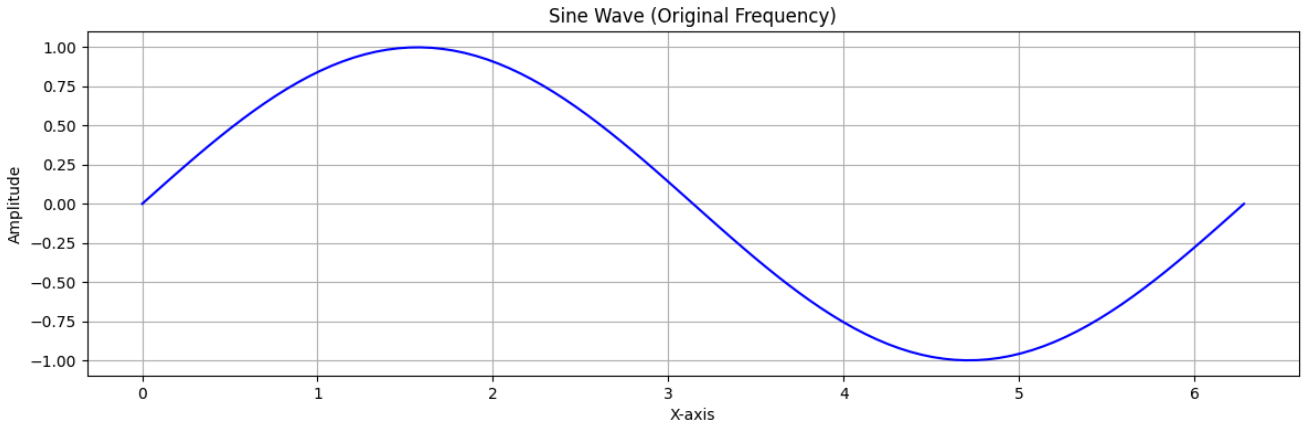
# Generate a different sine wave signal for the second subplot (e.g., higher frequency)
sine_wave_2 = np.sin(3 * x_values) # Modified frequency

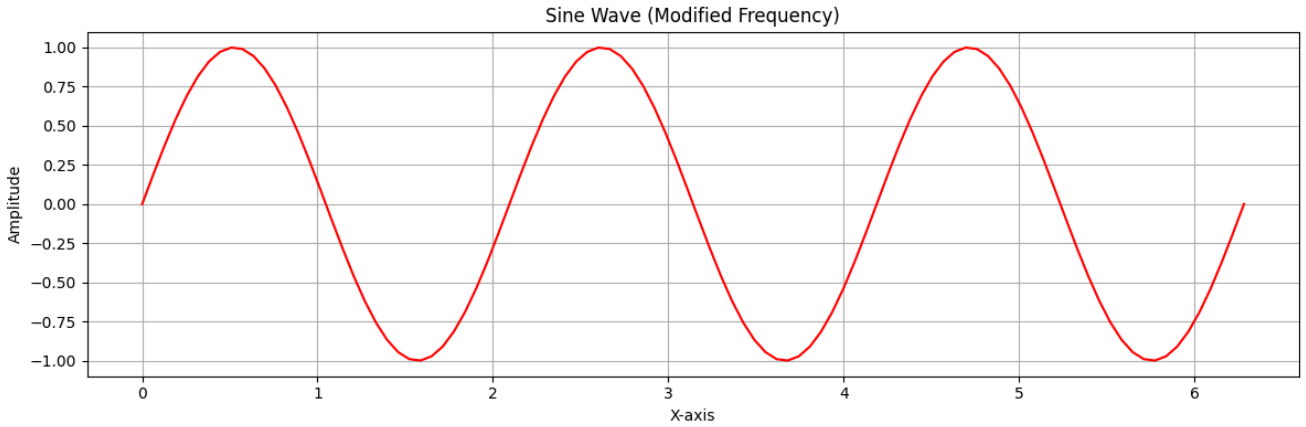
# Create a figure with two subplots
plt.figure(figsize=(12, 8)) # Increased figure height for vertical layout

# First subplot: Sine Wave (original frequency)
plt.subplot(2, 1, 1) # 2 rows, 1 column, first plot (top)
plt.plot(x_values, sine_wave_1, color='blue')
plt.title("Sine Wave (Original Frequency)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Second subplot: Modified Sine Wave
plt.subplot(2, 1, 2) # 2 rows, 1 column, second plot (bottom)
plt.plot(x_values, sine_wave_2, color='red') # Changed color
plt.title("Sine Wave (Modified Frequency)") # Changed title
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)


plt.tight_layout() # Adjust layout to prevent overlapping titles/labels
plt.show()
```






```
import numpy as np
import matplotlib.pyplot as plt

# Generate x values
x_values = np.linspace(0, 2 * np.pi, 100)

# Generate two signals
signal1 = np.sin(x_values)
signal2 = np.sin(3 * x_values)

# Add and subtract signals
sum_signal = signal1 + signal2
diff_signal = signal1 - signal2

# Create figure with 4 subplots
plt.figure(figsize=(12, 12))

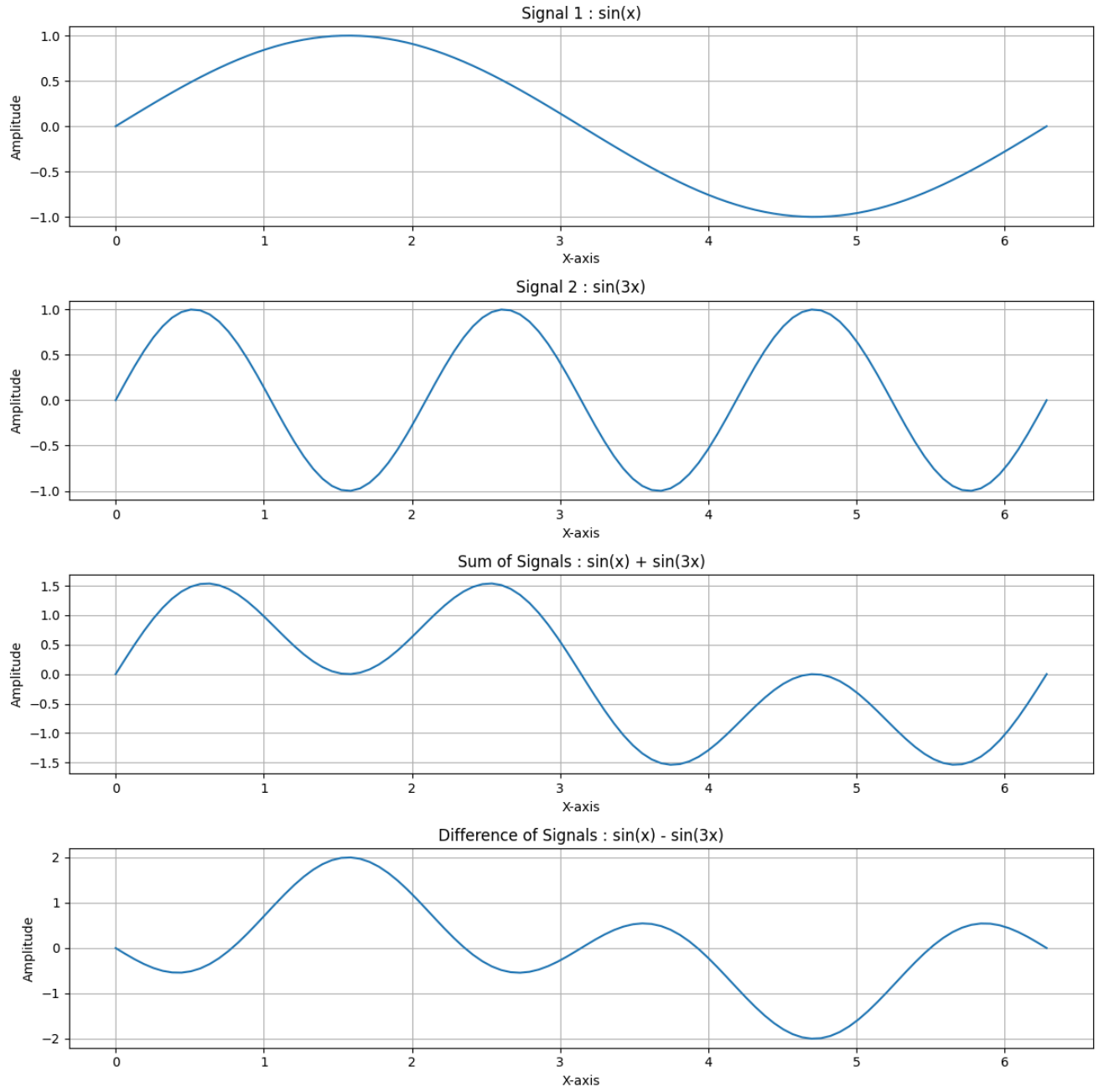
# Subplot 1: Signal 1
plt.subplot(4, 1, 1)
plt.plot(x_values, signal1)
plt.title("Signal 1 : sin(x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Subplot 2: Signal 2
plt.subplot(4, 1, 2)
plt.plot(x_values, signal2)
plt.title("Signal 2 : sin(3x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)
```

```
# Subplot 3: Sum of signals
plt.subplot(4, 1, 3)
plt.plot(x_values, sum_signal)
plt.title("Sum of Signals : sin(x) + sin(3x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Subplot 4: Difference of signals
plt.subplot(4, 1, 4)
plt.plot(x_values, diff_signal)
plt.title("Difference of Signals : sin(x) - sin(3x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

plt.tight_layout()
plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt

# Generate x values
```

```
x_values = np.linspace(0, 2 * np.pi, 100)

# Generate two signals
signal1 = np.sin(x_values)
signal2 = np.sin(3 * x_values)

# Add and subtract signals
sum_signal = signal1 + signal2
diff_signal = signal1 - signal2

# Create figure with 4 subplots
plt.figure(figsize=(12, 12))

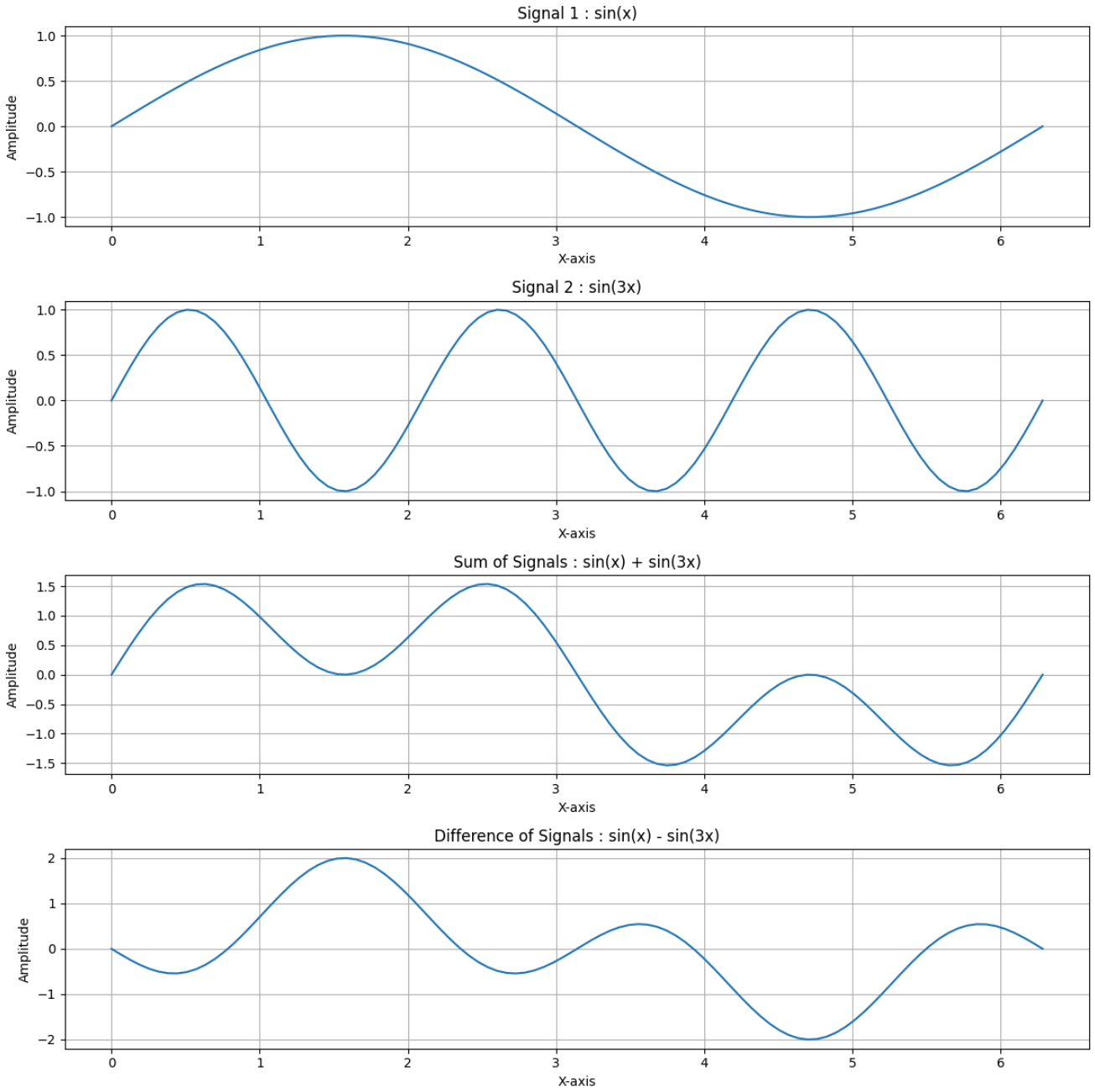
# Subplot 1: Signal 1
plt.subplot(4, 1, 1)
plt.plot(x_values, signal1)
plt.title("Signal 1 : sin(x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

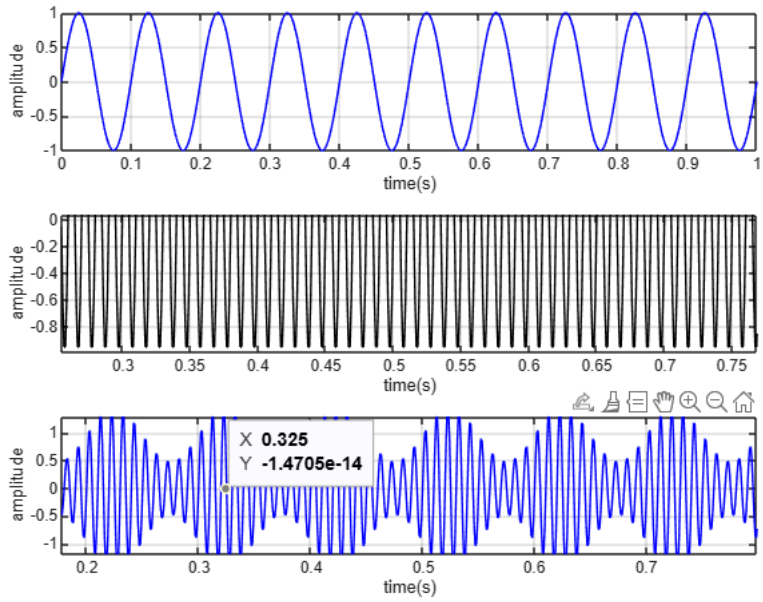
# Subplot 2: Signal 2
plt.subplot(4, 1, 2)
plt.plot(x_values, signal2)
plt.title("Signal 2 : sin(3x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Subplot 3: Sum of signals
plt.subplot(4, 1, 3)
plt.plot(x_values, sum_signal)
plt.title("Sum of Signals : sin(x) + sin(3x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Subplot 4: Difference of signals
plt.subplot(4, 1, 4)
plt.plot(x_values, diff_signal)
plt.title("Difference of Signals : sin(x) - sin(3x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

plt.tight_layout()
plt.show()
```


Signal 1 : sin(x)
Amplitude
X-axis
Signal 2 : sin(3x)
Amplitude
X-axis
Sum of Signals : sin(x) + sin(3x)
Amplitude
X-axis
Difference of Signals : sin(x) - sin(3x)
Amplitude
X-axis





```
import ipywidgets as widgets
from IPython.display import display
import random

btns=widgets.ToggleButtons(
    options = ['rock','paper','scissors'],
    description='Choose:'
)
out=widgets.Label()
def play(change):
  comp = random.choice(['rock','paper','scissors'])
  user = btns.value
  if user == comp:
      out.value = f"Tie! Both chose {comp}"
  elif(user=='rock' and comp=='scissors') or (user=='paper' and comp=='rock') or (user=='scissors' and comp=='paper'):
    out.value = f"You win! {user} Computer chose {comp}"
  else:
    out.value = f"You lose! {user} Computer chose {comp}"
btns.observe(play,names='value')
display(btns,out)
```

Choose: rock paper scissors

You win! paper Computer chose rock

```
import ipywidgets as widgets
from IPython.display import display
import random

btns=widgets.ToggleButtons(
    options = ['rock','paper','scissors'],
    description='Choose:'
)
out=widgets.Label()
def play(change):
  comp = random.choice(['rock','paper','scissors'])
  user = btns.value
  if user == comp:
      out.value = f"Tie! Both chose {comp}"
  elif(user=='rock' and comp=='scissors') or (user=='paper' and comp=='rock') or (user=='scissors' and comp=='paper'):
    out.value = f"You win! {user} Computer chose {comp}"
  else:
    out.value = f"You lose! {user} Computer chose {comp}"
btns.observe(play,names='value')
display(btns,out)
```

Choose: rock paper scissors

You win! paper Computer chose rock

```
import ipywidgets as widgets
from IPython.display import display, clear_output
import random

# Create a Button widget for rolling the dice
dice_button = widgets.Button(description="Roll Dice")

# Create an Output widget to display the dice roll result
dice_output = widgets.Output()

# Function to simulate a dice roll and display the result
def roll_dice(b):
    with dice_output:
        clear_output()
        roll = random.randint(1, 6)
        print(f"You rolled a: {roll}")

# Attach the roll_dice function to the button's on_click event
dice_button.on_click(roll_dice)

# Display the button and the output widget
display(dice_button, dice_output)
```

Roll Dice

```
You rolled a: 1
```

```
import ipywidgets as widgets
import random
from IPython.display import display, clear_output

secret = random.randint(1, 10)
output = widgets.Output()
slider = widgets.IntSlider(min=1, max=10, description='Guess:')

def on_change(change):
    with output:
        clear_output()
        if change['new'] == secret:
            print("You win!")
        elif change['new'] < secret:
            print("Too low!")
        else:
            print("Too high!")
```

```
slider.observe(on_change, names='value')
display(slider, output)
```

Guess: 1

```
Too low!
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
```

```
tips = sns.load_dataset('tips')
```

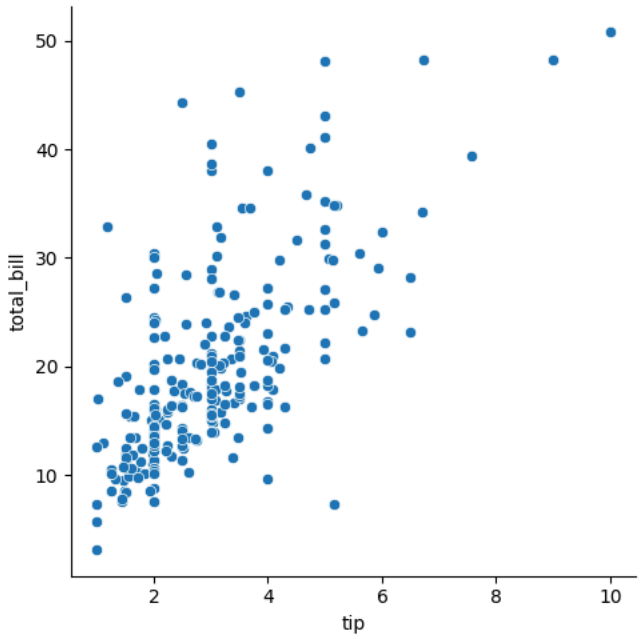
```
tips
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

244 rows × 7 columns

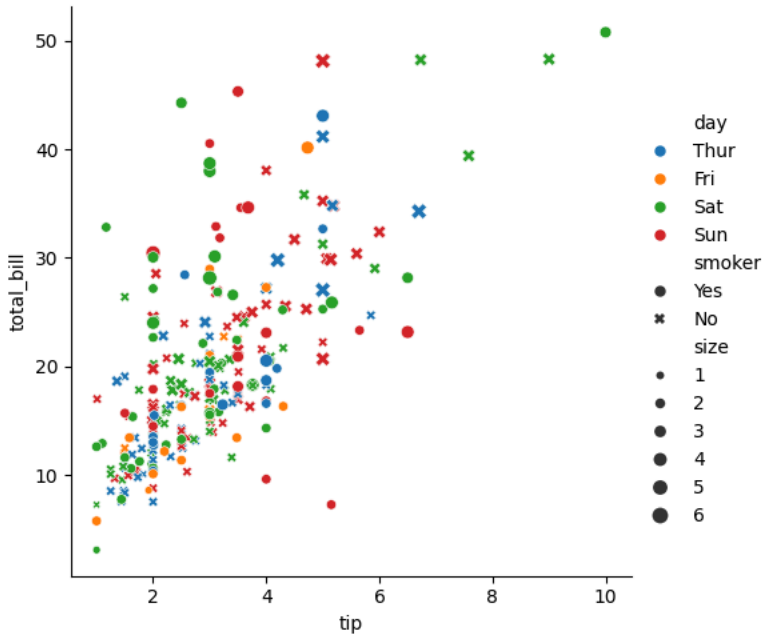
```
sns.relplot(x='tip',y='total_bill',data=tips)
```

```
<seaborn.axisgrid.FacetGrid at 0x7eb5b1f23bf0>
```



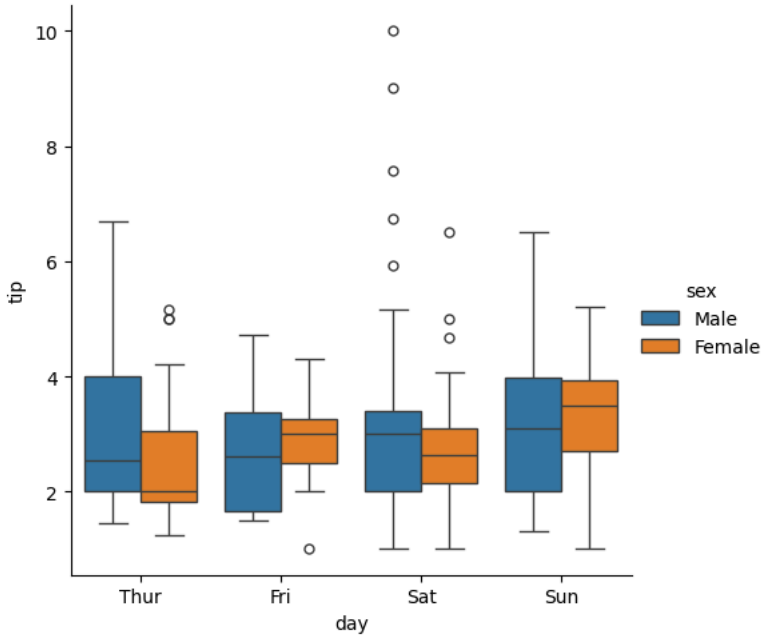

```
sns.relplot(x='tip',y='total_bill',data=tips,hue='day',style='smoker',size='size')
```

```
<seaborn.axisgrid.FacetGrid at 0x7eb5b17a0cb0>
```




```
sns.catplot(x='day',y='tip',data=tips,kind='box',hue='sex')
```

```
<seaborn.axisgrid.FacetGrid at 0x7eb5b140e570>
```




```
import numpy as np
import matplotlib.pyplot as plt

# Generate x values
x_values = np.linspace(0, 2 * np.pi, 100)

# Generate sine wave signal
sine_wave = np.sin(x_values)

# Generate cosine wave signal
cos_wave = np.cos(x_values)

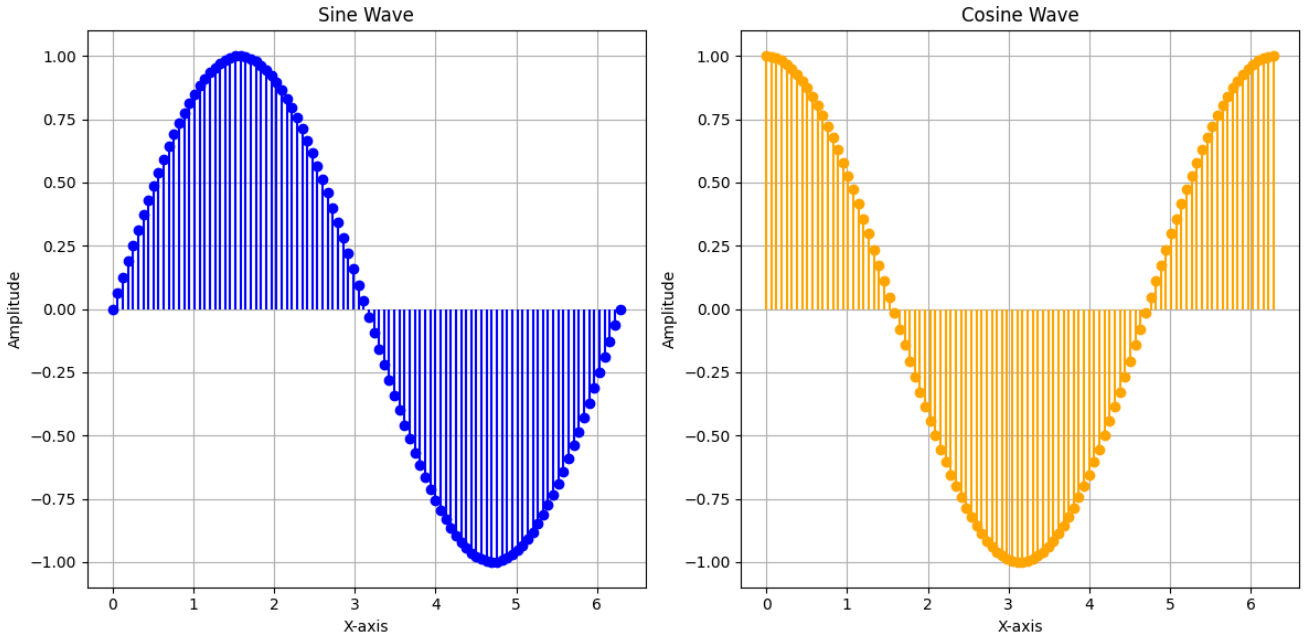
# Create a figure with two subplots
plt.figure(figsize=(12, 6))

# First subplot: Sine Wave
plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
plt.stem(x_values, sine_wave, linefmt='blue', markerfmt='ob', basefmt=' ') # Changed to plt.stem
```

```
plt.title("Sine Wave")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Second subplot: Cosine Wave
plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
plt.stem(x_values, cos_wave, linefmt='orange', markerfmt='o', basefmt=' ') # Changed to plt.stem
plt.title("Cosine Wave")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

plt.tight_layout() # Adjust layout to prevent overlapping titles/labels
plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt

# Generate x values
x_values = np.linspace(0, 2 * np.pi, 100)

# Generate sine wave signal for the first subplot
sine_wave_1 = np.sin(x_values)

# Generate a different sine wave signal for the second subplot (e.g., higher frequency)
sine_wave_2 = np.sin(3 * x_values) # Modified frequency

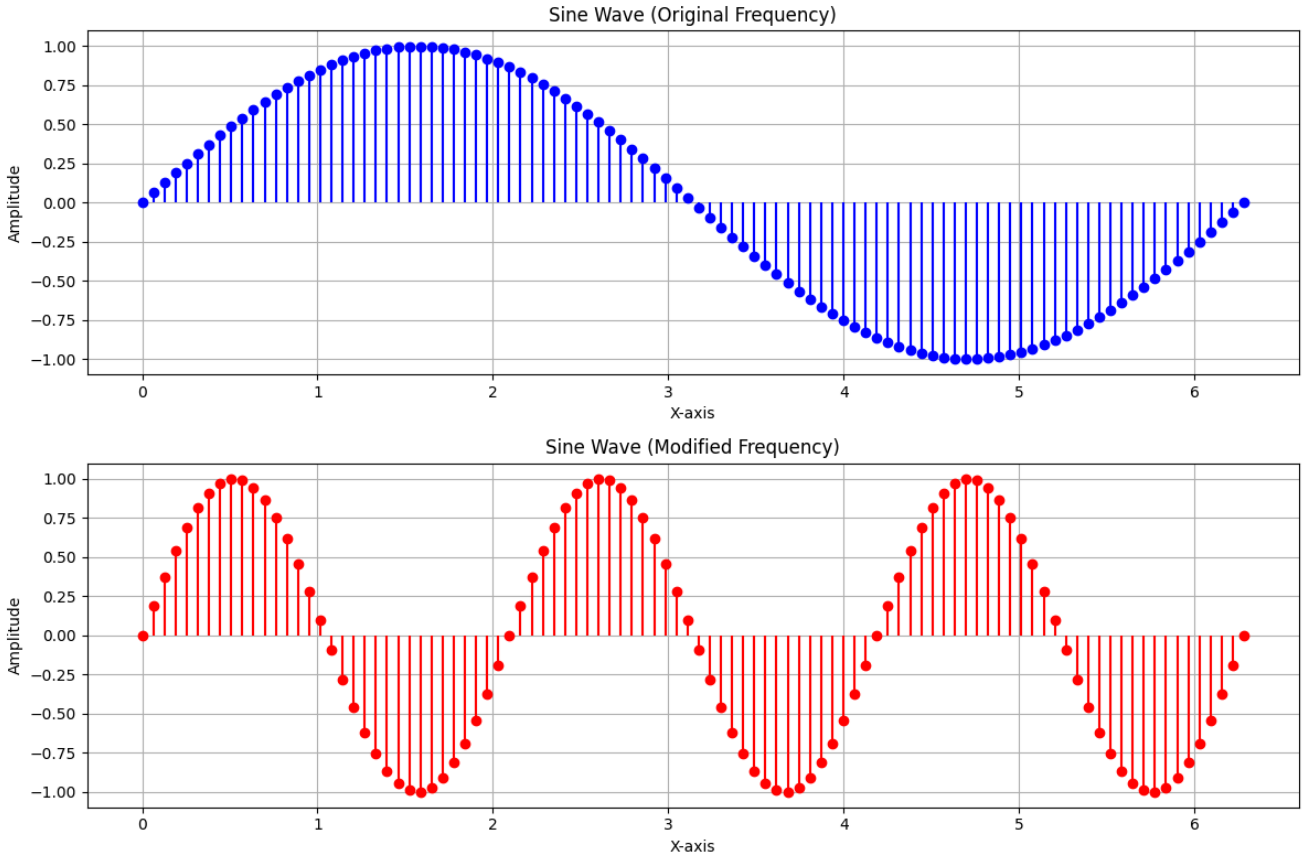
# Create a figure with two subplots
plt.figure(figsize=(12, 8)) # Increased figure height for vertical layout

# First subplot: Sine Wave (original frequency)
plt.subplot(2, 1, 1) # 2 rows, 1 column, first plot (top)
plt.stem(x_values, sine_wave_1, linefmt='blue', markerfmt='ob', basefmt=' ') # Changed to plt.stem
plt.title("Sine Wave (Original Frequency)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Second subplot: Modified Sine Wave
plt.subplot(2, 1, 2) # 2 rows, 1 column, second plot (bottom)
plt.stem(x_values, sine_wave_2, linefmt='red', markerfmt='o', basefmt=' ') # Changed to plt.stem
plt.title("Sine Wave (Modified Frequency)") # Changed title
plt.xlabel("X-axis")
```

```
plt.ylabel("Amplitude")
plt.grid(True)


plt.tight_layout() # Adjust layout to prevent overlapping titles/labels
plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt

# Generate x values
x_values = np.linspace(0, 2 * np.pi, 100)

# Generate sine wave signal for the first subplot
sine_wave_1 = np.sin(x_values)

# Generate a different sine wave signal for the second subplot (e.g., higher frequency)
sine_wave_2 = np.sin(3 * x_values) # Modified frequency

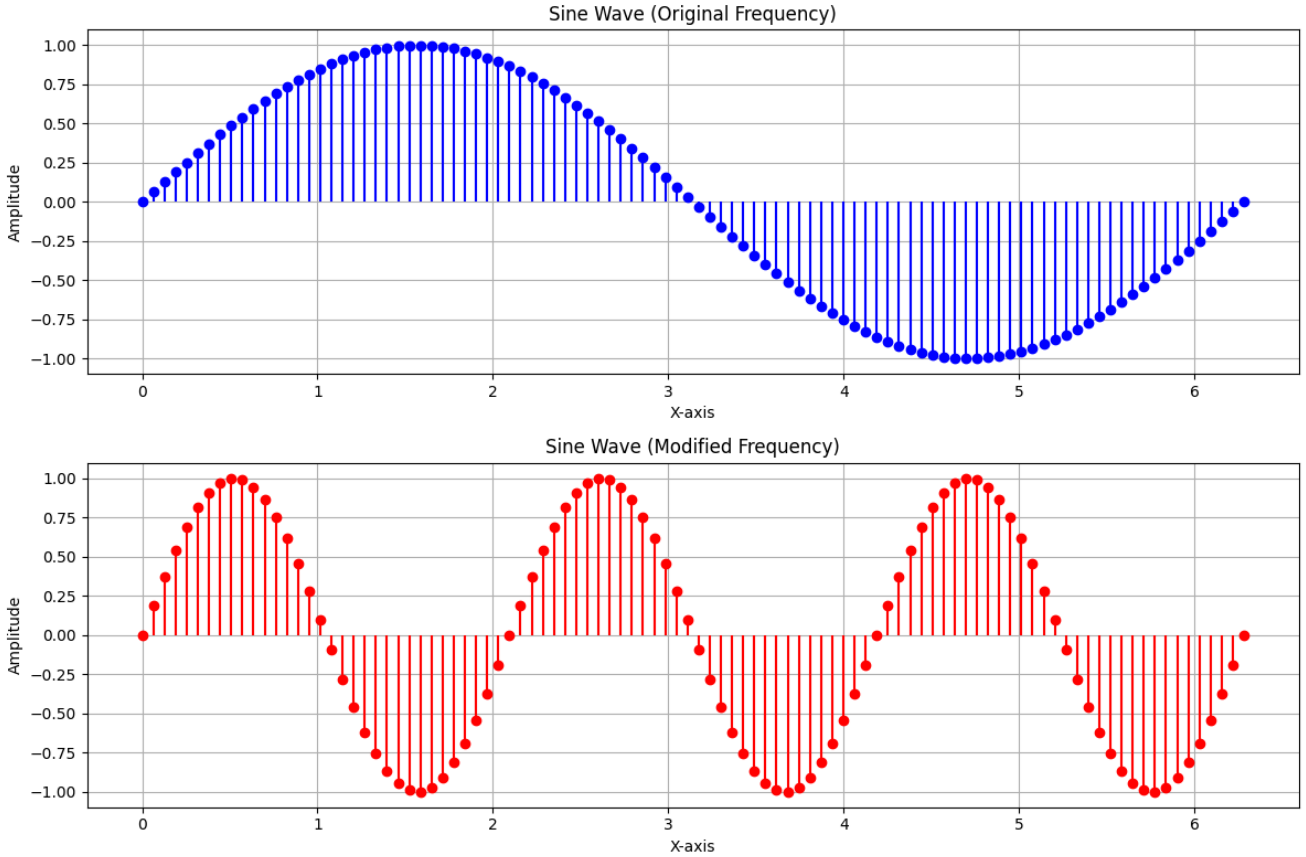
# Create a figure with two subplots
plt.figure(figsize=(12, 8)) # Increased figure height for vertical layout

# First subplot: Sine Wave (original frequency)
plt.subplot(2, 1, 1) # 2 rows, 1 column, first plot (top)
plt.stem(x_values, sine_wave_1, linefmt='blue', markerfmt='ob', basefmt=' ') # Changed to plt.stem
plt.title("Sine Wave (Original Frequency)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Second subplot: Modified Sine Wave
plt.subplot(2, 1, 2) # 2 rows, 1 column, second plot (bottom)
plt.stem(x_values, sine_wave_2, linefmt='red', markerfmt='o', basefmt=' ') # Changed to plt.stem
plt.title("Sine Wave (Modified Frequency)") # Changed title
```

```
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)


plt.tight_layout() # Adjust layout to prevent overlapping titles/labels
plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt

# Generate x values
x_values = np.linspace(0, 2 * np.pi, 100)

# Generate two signals
signal1 = np.sin(x_values)
signal2 = np.sin(3 * x_values)

# Add and subtract signals
sum_signal = signal1 + signal2
diff_signal = signal1 - signal2

# Create figure with 4 subplots
plt.figure(figsize=(12, 12))

# Subplot 1: Signal 1
plt.subplot(4, 1, 1)
plt.stem(x_values, signal1, linefmt='blue', markerfmt='ob', basefmt=' ') # Changed to plt.stem
plt.title("Signal 1 : sin(x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

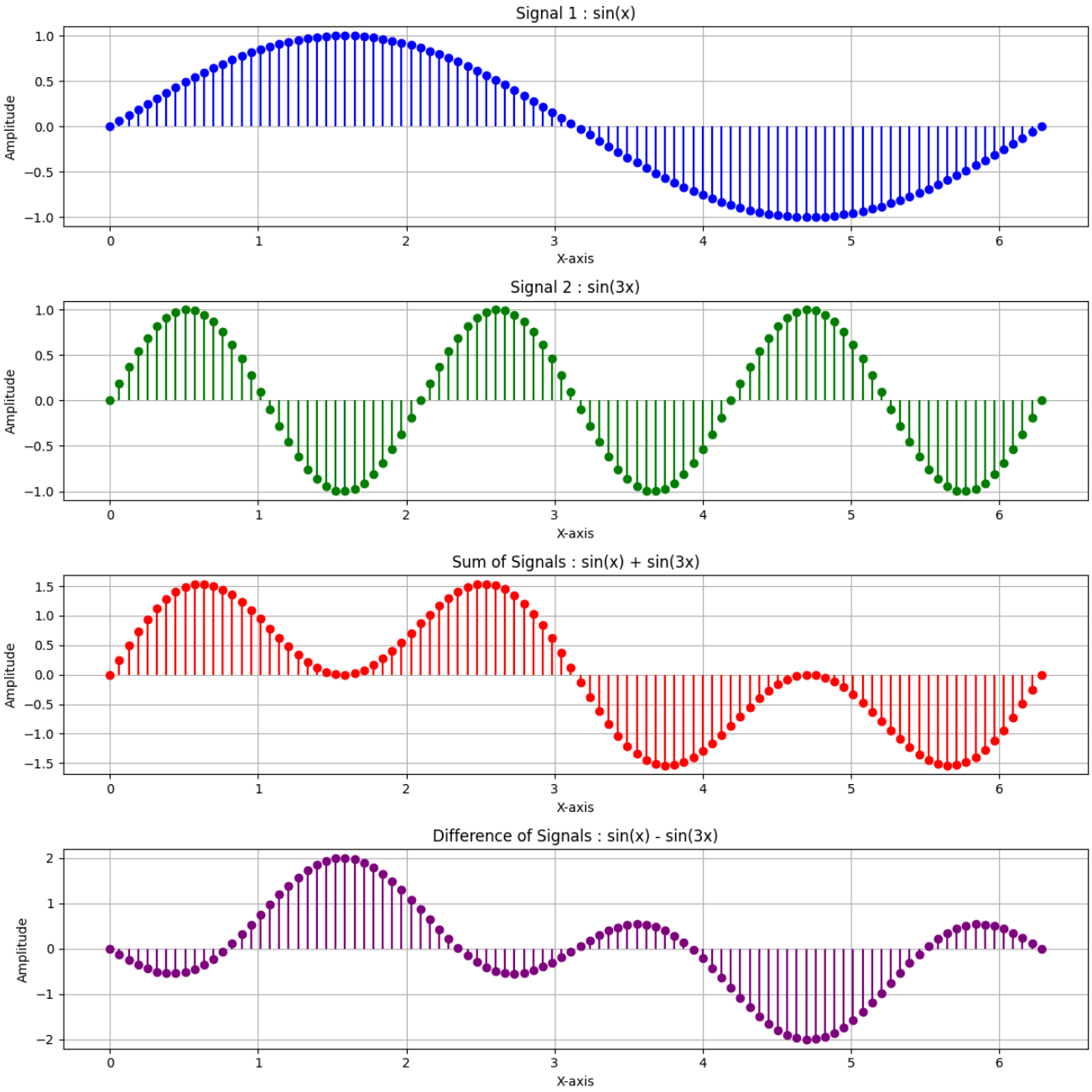
# Subplot 2: Signal 2
```

```
plt.subplot(4, 1, 2)
plt.stem(x_values, signal2, linefmt='green', markerfmt='og', basefmt=' ') # Changed to plt.stem
plt.title("Signal 2 : sin(3x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Subplot 3: Sum of signals
plt.subplot(4, 1, 3)
plt.stem(x_values, sum_signal, linefmt='red', markerfmt='or', basefmt=' ') # Changed to plt.stem
plt.title("Sum of Signals : sin(x) + sin(3x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Subplot 4: Difference of signals
plt.subplot(4, 1, 4)
plt.stem(x_values, diff_signal, linefmt='purple', markerfmt='o', basefmt=' ') # Changed to plt.stem
plt.title("Difference of Signals : sin(x) - sin(3x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

plt.tight_layout()
plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt

# Generate x values (already defined in previous subtask)
x_values = np.linspace(0, 2 * np.pi, 100)

# Generate signals (already defined in previous subtask)
signal1 = np.sin(x_values)
signal2 = np.sin(3 * x_values)

# Add and subtract signals (already defined in previous subtask)
sum_signal = signal1 + signal2
diff_signal = signal1 - signal2

# Create figure with 4 subplots in a 2x2 layout
plt.figure(figsize=(10, 8)) # Adjusted figsize for a 2x2 layout

# Subplot 1: Signal 1
plt.subplot(2, 2, 1) # 2 rows, 2 columns, first plot
plt.stem(x_values, signal1, linefmt='blue', markerfmt='ob', basefmt=' ') # Changed to plt.stem
plt.title("Signal 1 : sin(x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
```

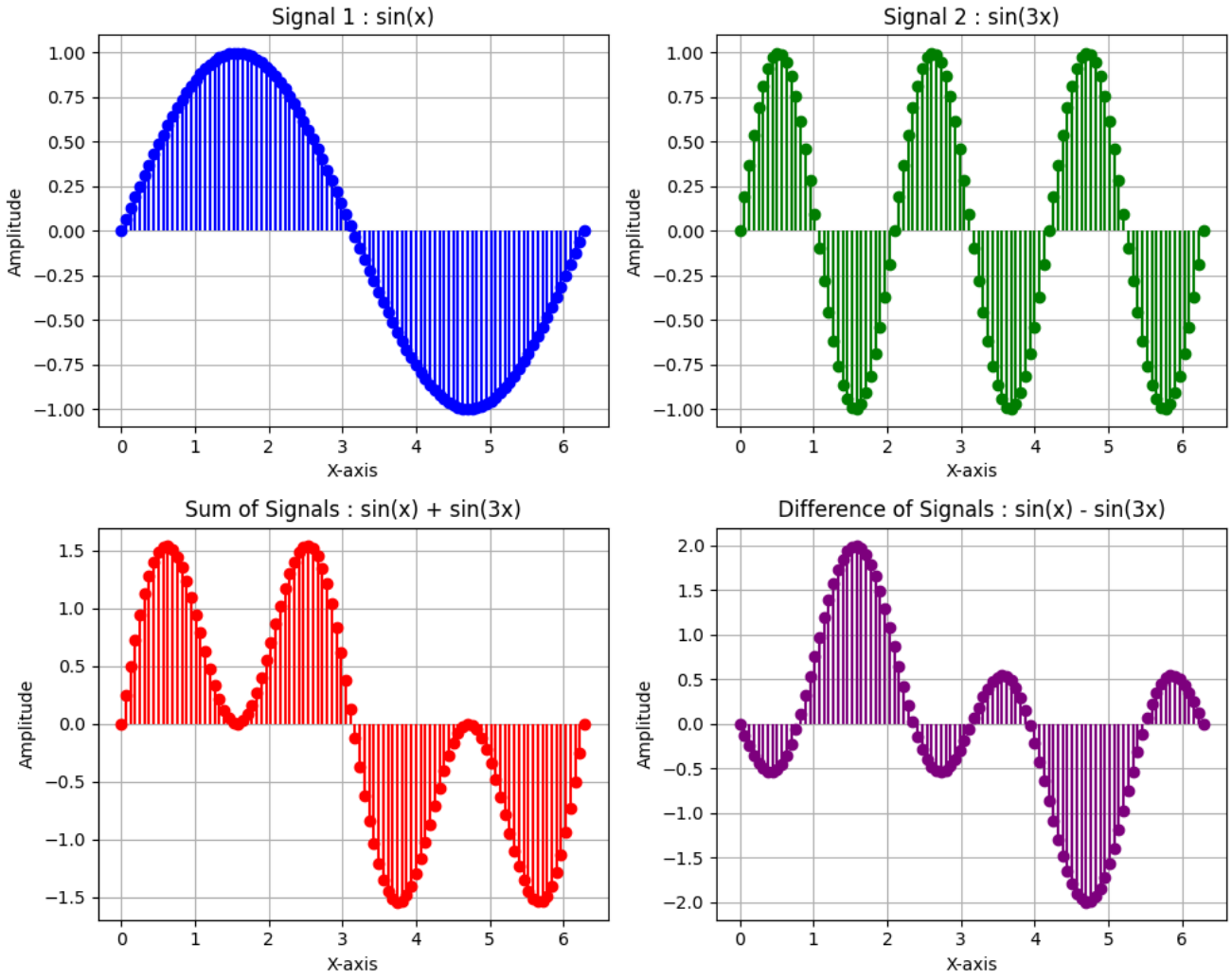
```
plt.grid(True)

# Subplot 2: Signal 2
plt.subplot(2, 2, 2) # 2 rows, 2 columns, second plot
plt.stem(x_values, signal2, linefmt='green', markerfmt='og', basefmt=' ') # Changed to plt.stem
plt.title("Signal 2 : sin(3x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Subplot 3: Sum of signals
plt.subplot(2, 2, 3) # 2 rows, 2 columns, third plot
plt.stem(x_values, sum_signal, linefmt='red', markerfmt='or', basefmt=' ') # Changed to plt.stem
plt.title("Sum of Signals : sin(x) + sin(3x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Subplot 4: Difference of signals
plt.subplot(2, 2, 4) # 2 rows, 2 columns, fourth plot
plt.stem(x_values, diff_signal, linefmt='purple', markerfmt='o', basefmt=' ') # Changed to plt.stem
plt.title("Difference of Signals : sin(x) - sin(3x)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

plt.tight_layout() # Adjust layout to prevent overlapping titles/labels
plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt

# Generate x values
x_values = np.linspace(0, 2 * np.pi, 100)
```

```
# Generate signals
sine_wave_1 = np.sin(x_values)
sine_wave_2 = np.sin(3 * x_values)

# Add the two signals
sum_signal = sine_wave_1 + sine_wave_2

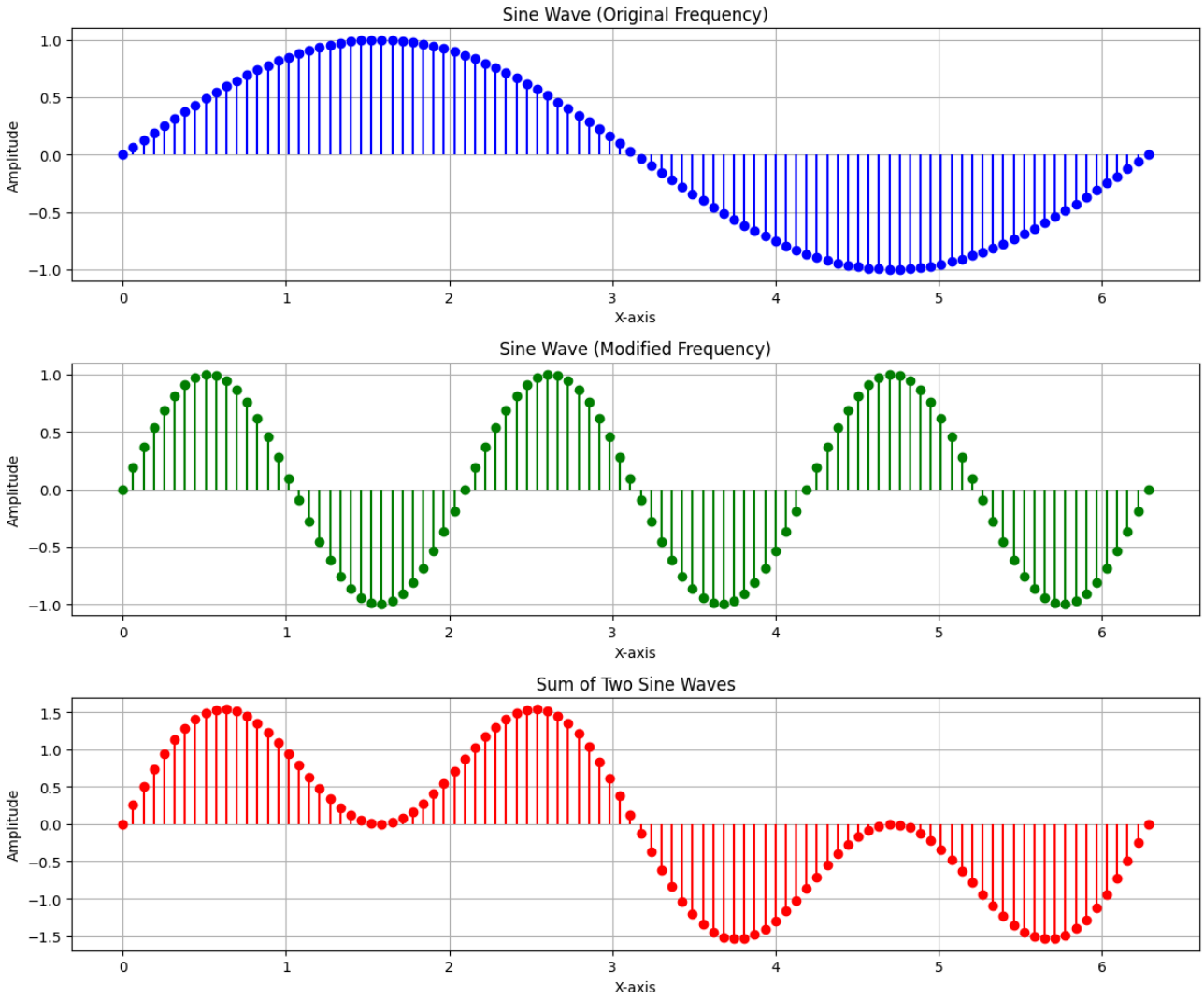
# Create figure with 3 subplots
plt.figure(figsize=(12, 10))

# Subplot 1: Original sine wave
plt.subplot(3, 1, 1)
plt.stem(x_values, sine_wave_1, linefmt='blue', markerfmt='ob', basefmt=' ') # Changed to plt.stem
plt.title("Sine Wave (Original Frequency)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Subplot 2: Modified sine wave
plt.subplot(3, 1, 2)
plt.stem(x_values, sine_wave_2, linefmt='green', markerfmt='og', basefmt=' ') # Changed to plt.stem
plt.title("Sine Wave (Modified Frequency)")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

# Subplot 3: Sum of both signals
plt.subplot(3, 1, 3)
plt.stem(x_values, sum_signal, linefmt='red', markerfmt='or', basefmt=' ') # Changed to plt.stem
plt.title("Sum of Two Sine Waves")
plt.xlabel("X-axis")
plt.ylabel("Amplitude")
plt.grid(True)

plt.tight_layout()
plt.show()
```


Sine Wave (Original Frequency)
Amplitude
X-axis
Sine Wave (Modified Frequency)
Amplitude
X-axis
Sum of Two Sine Waves
Amplitude
X-axis





```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

# Load the image
img = mpimg.imread('/content/images.jpg')

# Display the image
plt.imshow(img)
plt.axis('off') # Turn off axis numbers and ticks
plt.title('My Image Plot') # Add a title to the image
plt.show()
```




```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

# Load the image
img = mpimg.imread('/content/bird-breed.jpg')

# Display the image
plt.imshow(img)
plt.axis('off') # Turn off axis numbers and ticks
plt.title('My Image Plot') # Add a title to the image
plt.show()
```




```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

# Load the image from the Colab environment
img = mpimg.imread('/content/Parakeets.jpeg')

# Display the image
plt.imshow(img)
plt.axis('off') # Turn off axis numbers and ticks
plt.title('My Parakeets Image') # Add a title to the image
plt.show()
```




```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

# Load the images
img1 = mpimg.imread('/content/Parakeets.jpeg')
img2 = mpimg.imread('/content/images.jpg')
img3 = mpimg.imread('/content/bird-breed.jpg')

# Create a figure with three subplots
plt.figure(figsize=(15, 5))

# First subplot
plt.subplot(1, 3, 1) # 1 row, 3 columns, first plot
plt.imshow(img1)
plt.axis('off') # Turn off axis numbers and ticks
plt.title('Parakeets Image') # Add a title to the image

# Second subplot
plt.subplot(1, 3, 2) # 1 row, 3 columns, second plot
plt.imshow(img2)
plt.axis('off') # Turn off axis numbers and ticks
plt.title('General Image') # Add a title to the image

# Third subplot
plt.subplot(1, 3, 3) # 1 row, 3 columns, third plot
plt.imshow(img3)
plt.axis('off') # Turn off axis numbers and ticks
plt.title('Bird Breed Image') # Add a title to the image

plt.tight_layout() # Adjust layout to prevent overlapping titles/labels
plt.show()
```




```
print("Numbers divisible by 6 from 1 to 100:")
for num in range(1, 101):
```

```
    if num % 6 == 0:
        print(num)
```

```
Numbers divisible by 6 from 1 to 100:
6
12
18
24
30
36
42
48
54
60
66
72
78
84
90
96
```

```
print("Prime numbers from 1 to 100:")
for num in range(1, 101):
    if num > 1:  # Prime numbers are greater than 1
        is_prime = True
        for i in range(2, int(num**0.5) + 1):
            if num % i == 0:
                is_prime = False
                break
        if is_prime:
            print(num)
```

```
Prime numbers from 1 to 100:
2
3
5
7
11
13
17
19
23
29
31
37
41
43
47
53
59
61
67
71
73
79
83
89
97
```

print the that are divisble by 6 that from 1 to 100 print 1to100 all numbers are prime numbers

plotting the following signals for plotting different signals A exoperntial B sine wave C cos wave D parabola E complex signal(sin+cos) F discrerte complex signal(sin+cos)

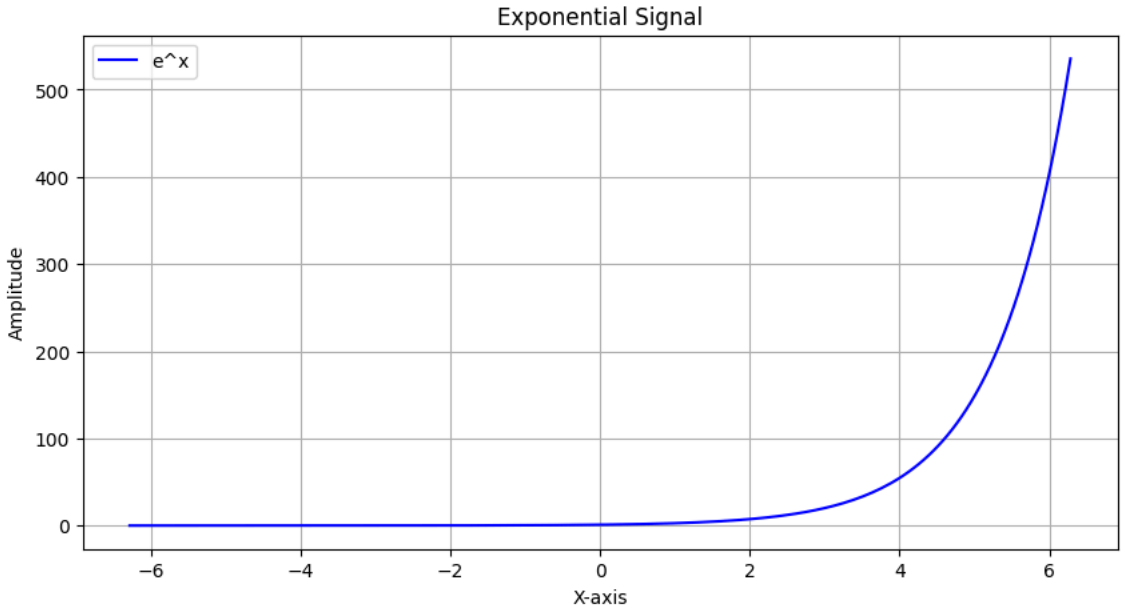
```
Start coding or generate with AI.
```

```
import numpy as np
import matplotlib.pyplot as plt

# Define common x values for continuous signals
x_values_continuous = np.linspace(-2 * np.pi, 2 * np.pi, 200)
x_values_discrete = np.arange(-2 * np.pi, 2 * np.pi, 0.5)
```

```
# A: Exponential Signal
plt.figure(figsize=(10, 5))
plt.plot(x_values_continuous, np.exp(x_values_continuous), label='e^x', color='blue')
```

```
plt.title('Exponential Signal')
plt.xlabel('X-axis')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()
plt.show()
```



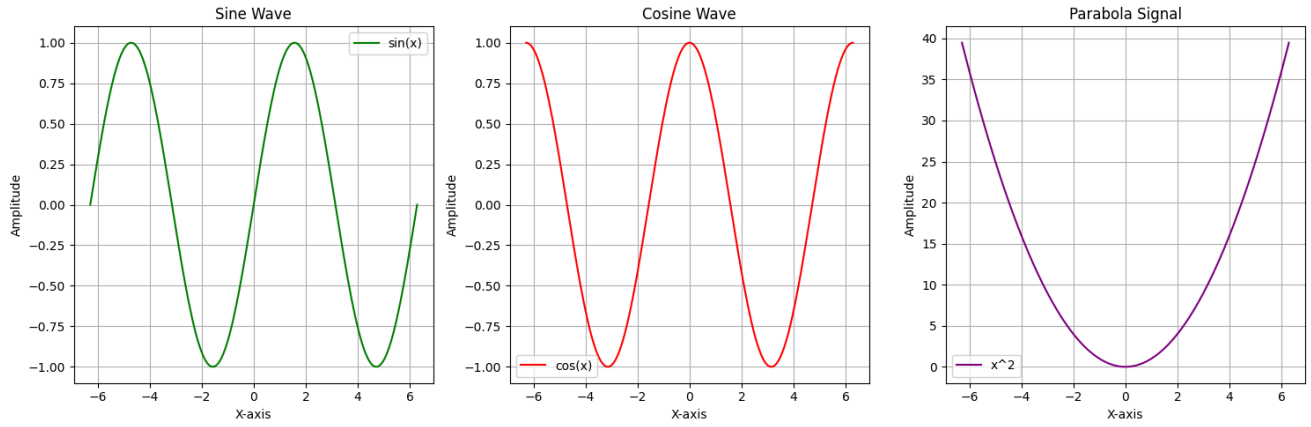

```
# B: Sine Wave, C: Cosine Wave, D: Parabola
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.plot(x_values_continuous, np.sin(x_values_continuous), label='sin(x)', color='green')
plt.title('Sine Wave')
plt.xlabel('X-axis')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()

plt.subplot(1, 3, 2)
plt.plot(x_values_continuous, np.cos(x_values_continuous), label='cos(x)', color='red')
plt.title('Cosine Wave')
plt.xlabel('X-axis')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()

plt.subplot(1, 3, 3)
plt.plot(x_values_continuous, x_values_continuous**2, label='x^2', color='purple')
plt.title('Parabola Signal')
plt.xlabel('X-axis')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()
```



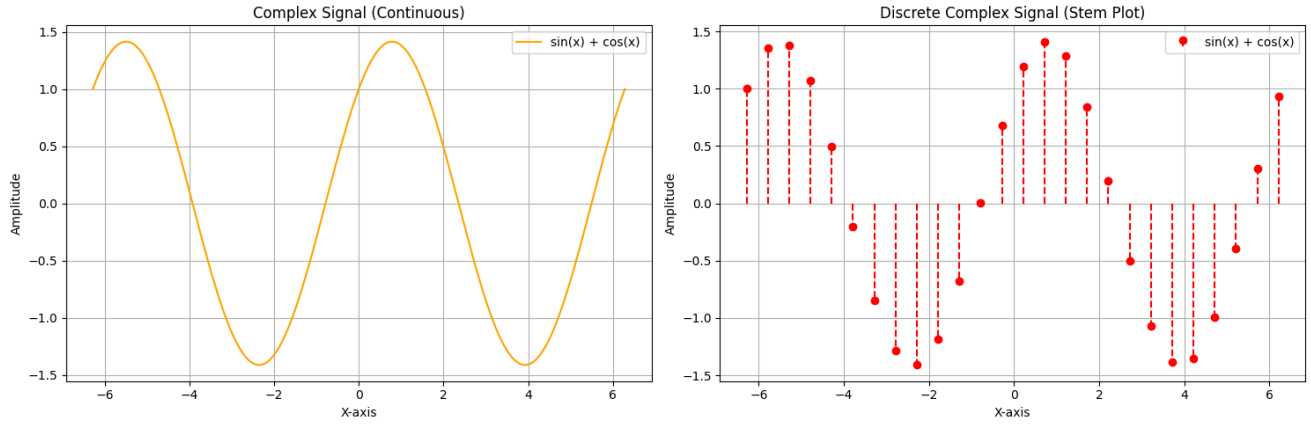

```
# E: Complex Signal (sin+cos) and F: Discrete Complex Signal (sin+cos)
complex_signal_continuous = np.sin(x_values_continuous) + np.cos(x_values_continuous)
complex_signal_discrete = np.sin(x_values_discrete) + np.cos(x_values_discrete)

plt.figure(figsize=(15, 5))

plt.subplot(1, 2, 1)
plt.plot(x_values_continuous, complex_signal_continuous, label='sin(x) + cos(x)', color='orange')
plt.title('Complex Signal (Continuous)')
plt.xlabel('X-axis')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()

plt.subplot(1, 2, 2)
plt.stem(x_values_discrete, complex_signal_discrete, linefmt='--r', markerfmt='or', basefmt=' ', label='sin(x) + cos(x)')
plt.title('Discrete Complex Signal (Stem Plot)')
plt.xlabel('X-axis')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt

# Define common x values for continuous and discrete signals
x_values_continuous = np.linspace(-2 * np.pi, 2 * np.pi, 200)
x_values_discrete = np.arange(-2 * np.pi, 2 * np.pi, 0.5)

# Calculate complex signals
complex_signal_continuous = np.sin(x_values_continuous) + np.cos(x_values_continuous)
complex_signal_discrete = np.sin(x_values_discrete) + np.cos(x_values_discrete)

# Create a figure with 2 rows and 3 columns for all 6 plots
plt.figure(figsize=(18, 12))

# Subplot A: Exponential Signal
plt.subplot(2, 3, 1) # 2 rows, 3 columns, 1st plot
plt.plot(x_values_continuous, np.exp(x_values_continuous), label='e^x', color='blue')
plt.title('A: Exponential Signal')
plt.xlabel('X-axis')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()

# Subplot B: Sine Wave
plt.subplot(2, 3, 2) # 2 rows, 3 columns, 2nd plot
plt.plot(x_values_continuous, np.sin(x_values_continuous), label='sin(x)', color='green')
plt.title('B: Sine Wave')
plt.xlabel('X-axis')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()

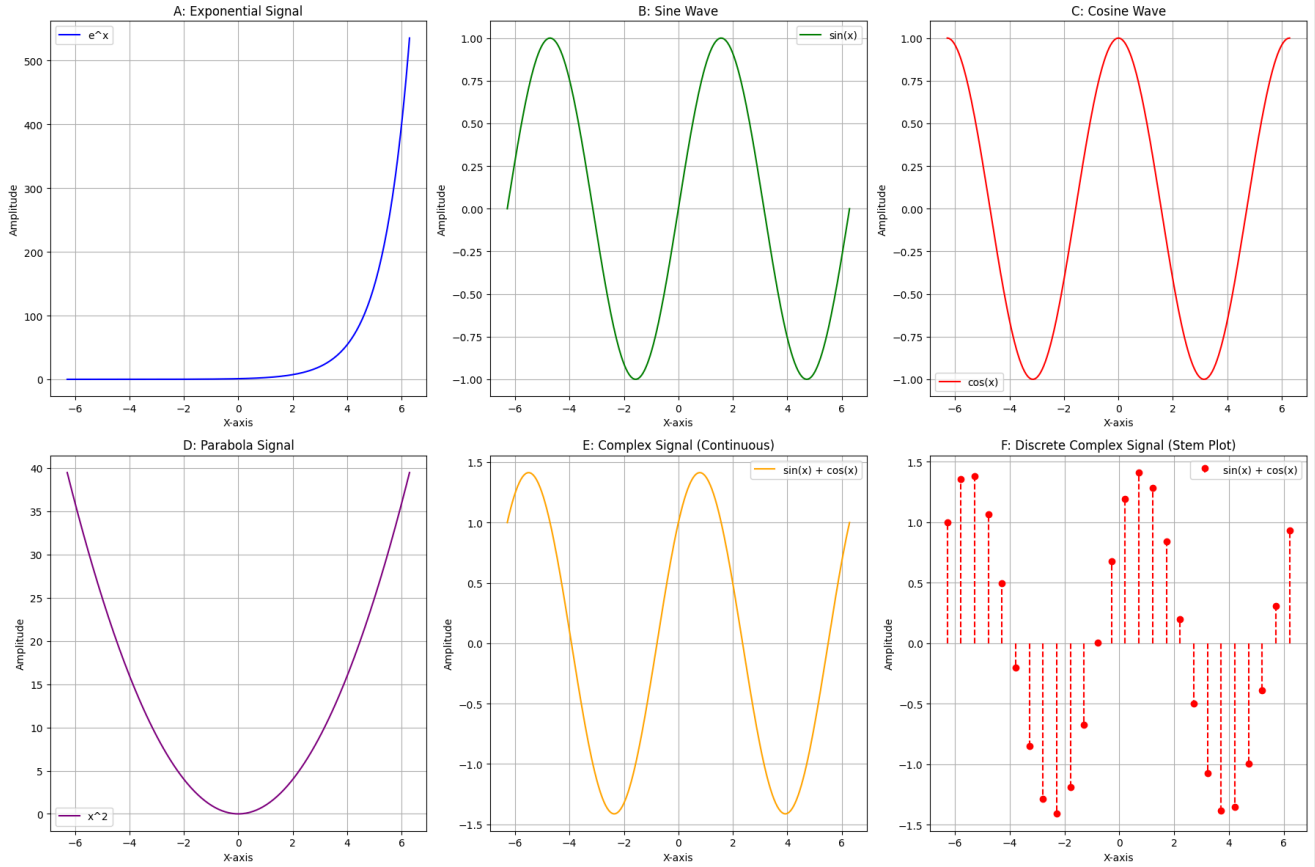
# Subplot C: Cosine Wave
plt.subplot(2, 3, 3) # 2 rows, 3 columns, 3rd plot
plt.plot(x_values_continuous, np.cos(x_values_continuous), label='cos(x)', color='red')
plt.title('C: Cosine Wave')
plt.xlabel('X-axis')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()

# Subplot D: Parabola
plt.subplot(2, 3, 4) # 2 rows, 3 columns, 4th plot
plt.plot(x_values_continuous, x_values_continuous**2, label='x^2', color='purple')
plt.title('D: Parabola Signal')
plt.xlabel('X-axis')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()

# Subplot E: Complex Signal (Continuous)
plt.subplot(2, 3, 5) # 2 rows, 3 columns, 5th plot
plt.plot(x_values_continuous, complex_signal_continuous, label='sin(x) + cos(x)', color='orange')
plt.title('E: Complex Signal (Continuous)')
plt.xlabel('X-axis')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()

# Subplot F: Discrete Complex Signal (Stem Plot)
plt.subplot(2, 3, 6) # 2 rows, 3 columns, 6th plot
plt.stem(x_values_discrete, complex_signal_discrete, linefmt='--r', markerfmt='or', basefmt=' ', label='sin(x) + cos(x)')
plt.title('F: Discrete Complex Signal (Stem Plot)')
plt.xlabel('X-axis')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()

plt.tight_layout() # Adjust layout to prevent overlapping titles/labels
plt.show()
```



Start coding or generate with AI.

```
road_signals = {
    'Red': ['R1', 'R2', 'R3'],
    'Green': ['G1', 'G2', 'G3'],
    'Blue': ['B1', 'B2', 'B3']
}

print("Defined Road Signals:")
for road, signals in road_signals.items():
    print(f"  {road} Road: {', '.join(signals)}")
```

```
Defined Road Signals:
  Red Road: R1, R2, R3
  Green Road: G1, G2, G3
  Blue Road: B1, B2, B3
```

```
import numpy as np
import matplotlib.pyplot as plt

# Time axis
t = np.arange(0, 60, 0.1)

# Initialize all signals
R1 = np.ones_like(t)
```

```
G1 = np.zeros_like(t)
B1 = np.zeros_like(t)

R2 = np.ones_like(t)
G2 = np.zeros_like(t)
B2 = np.zeros_like(t)

R3 = np.ones_like(t)
G3 = np.zeros_like(t)
B3 = np.zeros_like(t)

# Phase durations
G = 6     # green
Y = 2     # yellow
cycle = 3*(G + Y)

for i in range(0, len(t)):
    time = t[i] % cycle

    if time < G:                    # Road 3 green
        R3[i]=0; G3[i]=1
    elif time < G+Y:                # Road 3 yellow
        R3[i]=0; B3[i]=1

    elif time < 2*G+Y:              # Road 1 green
        R1[i]=0; G1[i]=1
    elif time < 2*(G+Y):            # Road 1 yellow
        R1[i]=0; B1[i]=1

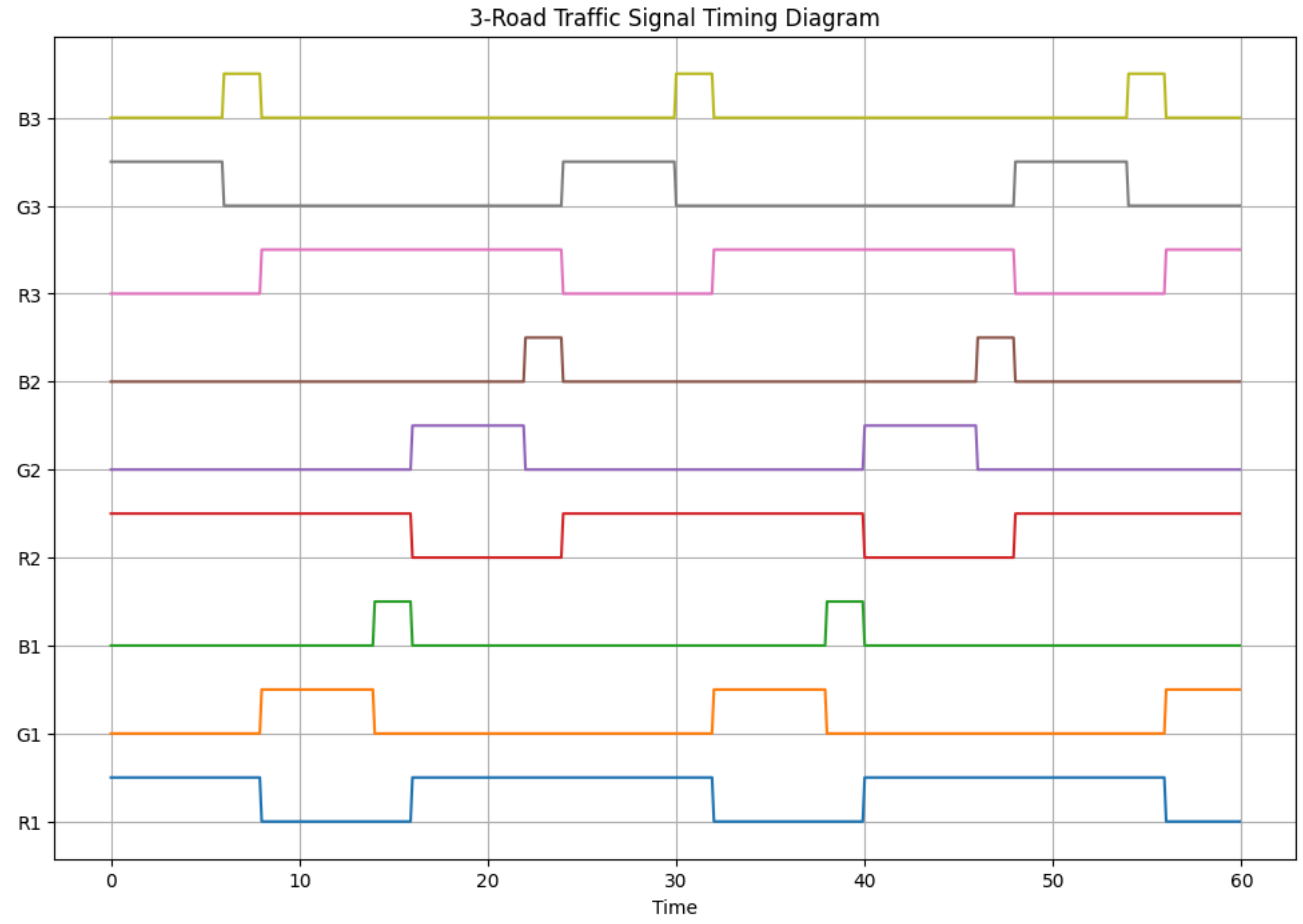
    elif time < 3*G+2*Y:            # Road 2 green
        R2[i]=0; G2[i]=1
    else:                           # Road 2 yellow
        R2[i]=0; B2[i]=1

# Plot waveforms
plt.figure(figsize=(12,8))

signals = [R1,G1,B1, R2,G2,B2, R3,G3,B3]
labels  = ['R1','G1','B1','R2','G2','B2','R3','G3','B3']

for i, sig in enumerate(signals):
    plt.plot(t, sig + 2*i)

plt.yticks(np.arange(0, 18, 2), labels)
plt.xlabel("Time")
plt.title("3-Road Traffic Signal Timing Diagram")
plt.grid(True)
plt.show()
```




```
road_signals = {
    'Red': ['R1', 'R2', 'R3'],
    'Green': ['G1', 'G2', 'G3'],
    'Blue': ['B1', 'B2', 'B3']
}

print("Defined Road Signals:")
for road, signals in road_signals.items():
    print(f"  {road} Road: {', '.join(signals)}")
```

```
Defined Road Signals:
  Red Road: R1, R2, R3
  Green Road: G1, G2, G3
  Blue Road: B1, B2, B3
```

```
import numpy as np
import matplotlib.pyplot as plt

# Time axis
t = np.arange(0, 60, 0.1)

# Initialize all signals
R1 = np.ones_like(t)
G1 = np.zeros_like(t)
B1 = np.zeros_like(t)

R2 = np.ones_like(t)
G2 = np.zeros_like(t)
B2 = np.zeros_like(t)

R3 = np.ones_like(t)
G3 = np.zeros_like(t)
B3 = np.zeros_like(t)

# Phase durations
G = 6     # green
Y = 2     # yellow
cycle = 3*(G + Y)
```

```
for i in range(0, len(t)):
    time = t[i] % cycle

    if time < G:                    # Road 3 green
        R3[i]=0; G3[i]=1
    elif time < G+Y:                # Road 3 yellow
        R3[i]=0; B3[i]=1

    elif time < 2*G+Y:              # Road 1 green
        R1[i]=0; G1[i]=1
    elif time < 2*(G+Y):            # Road 1 yellow
        R1[i]=0; B1[i]=1

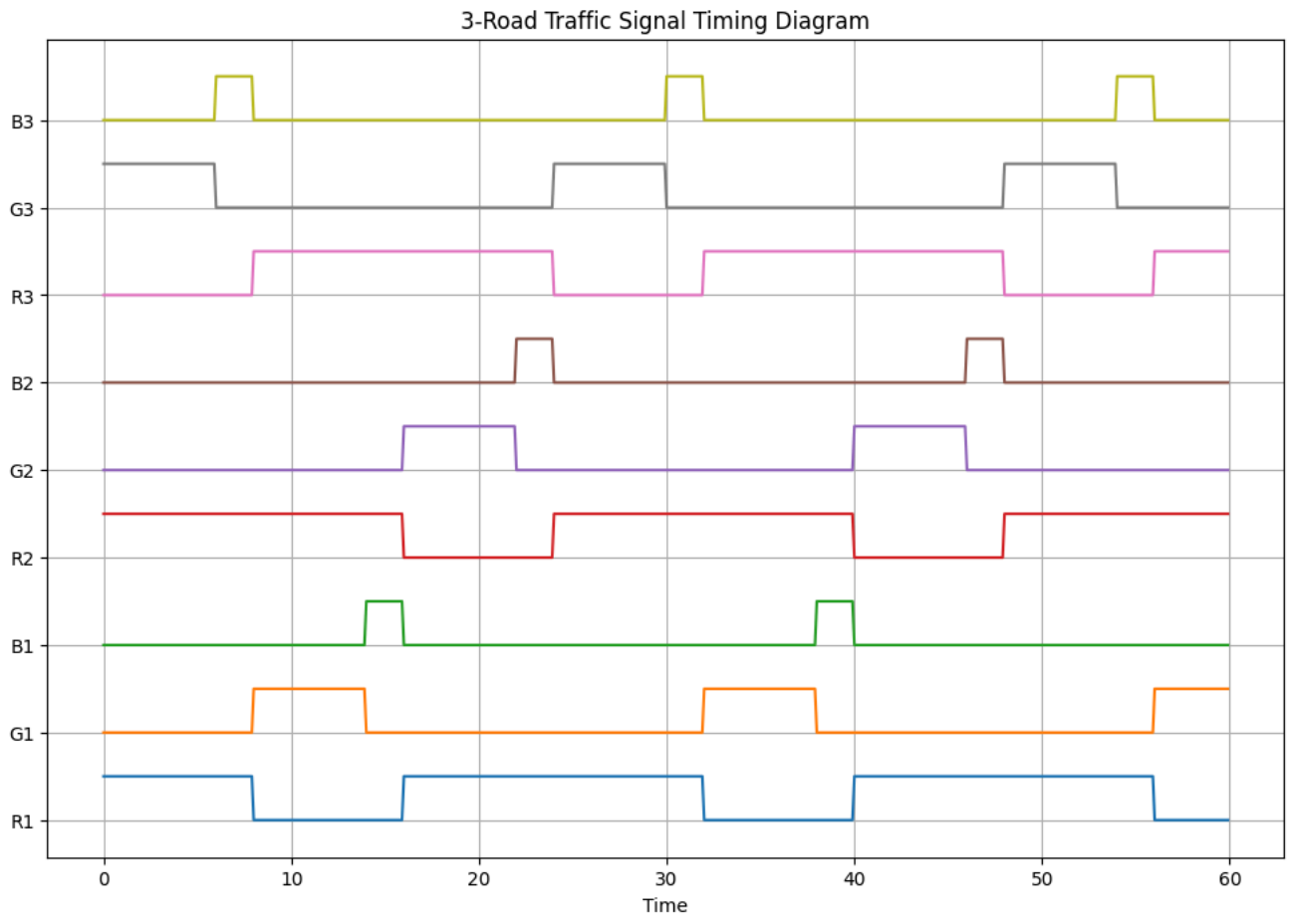
    elif time < 3*G+2*Y:            # Road 2 green
        R2[i]=0; G2[i]=1
    else:                           # Road 2 yellow
        R2[i]=0; B2[i]=1

# Plot waveforms
plt.figure(figsize=(12,8))

signals = [R1,G1,B1, R2,G2,B2, R3,G3,B3]
labels  = ['R1','G1','B1','R2','G2','B2','R3','G3','B3']

for i, sig in enumerate(signals):
    plt.plot(t, sig + 2*i)

plt.yticks(np.arange(0, 18, 2), labels)
plt.xlabel("Time")
plt.title("3-Road Traffic Signal Timing Diagram")
plt.grid(True)
plt.show()
```




```
!pip install -q streamlit
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.0/9.0 MB 54.5 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.9/6.9 MB 79.4 MB/s eta 0:00:00
```

```
!pip install streamlit localtunnel
!pip install streamlit
!npm install -g localtunnel
```

```
Requirement already satisfied: streamlit in /usr/local/lib/python3.12/dist-packages (1.52.2)
ERROR: Could not find a version that satisfies the requirement localtunnel (from versions: none)
ERROR: No matching distribution found for localtunnel
Requirement already satisfied: streamlit in /usr/local/lib/python3.12/dist-packages (1.52.2)
Requirement already satisfied: altair!=5.4.0,!=5.4.1,<7,>=4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (5.5.0
Requirement already satisfied: blinker<2,>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (1.9.0)
Requirement already satisfied: cachetools<7,>=4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (6.2.4)
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (8.3.1)
Requirement already satisfied: numpy<3,>=1.23 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.0.2)
Requirement already satisfied: packaging>=20 in /usr/local/lib/python3.12/dist-packages (from streamlit) (25.0)
Requirement already satisfied: pandas<3,>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.2.2)
Requirement already satisfied: pillow<13,>=7.1.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (11.3.0)
Requirement already satisfied: protobuf<7,>=3.20 in /usr/local/lib/python3.12/dist-packages (from streamlit) (5.29.5)
Requirement already satisfied: pyarrow>=7.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (18.1.0)
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.32.4)
Requirement already satisfied: tenacity<10,>=8.1.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (9.1.2)
Requirement already satisfied: toml<2,>=0.10.1 in /usr/local/lib/python3.12/dist-packages (from streamlit) (0.10.2)
Requirement already satisfied: typing-extensions<5,>=4.4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (4.15.0)
Requirement already satisfied: watchdog<7,>=2.1.5 in /usr/local/lib/python3.12/dist-packages (from streamlit) (6.0.0)
Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in /usr/local/lib/python3.12/dist-packages (from streamlit) (3.1.45)
Requirement already satisfied: pydeck<1,>=0.8.0b4 in /usr/local/lib/python3.12/dist-packages (from streamlit) (0.9.1)
Requirement already satisfied: tornado!=6.5.0,<7,>=6.0.3 in /usr/local/lib/python3.12/dist-packages (from streamlit) (6.5.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.0->streamlit
Requirement already satisfied: jsonschema>=3.0 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.0->
Requirement already satisfied: narwhals>=1.14.2 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.0-
Requirement already satisfied: gitdb<5,>=4.0.1 in /usr/local/lib/python3.12/dist-packages (from gitpython!=3.1.19,<4,>=3.0.7->st
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0->streaml
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0->streamlit) (2025.
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0->streamlit) (202
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->stre
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit) (3.11
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit)
Requirement already satisfied: smmap<6,>=3.0.1 in /usr/local/lib/python3.12/dist-packages (from gitdb<5,>=4.0.1->gitpython!=3.1.
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->altair!=5.4.0,!=5.4.1,<7
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!=5.4.0,!=5
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonschema>
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!=5.4
Requirement already satisfied: rpds-py>=0.7.1 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!=5.4.0,!=
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas<3,>=1.4.
⠙⠹ ⠸⠼⠴⠦⠧⠇
added 22 packages in 1s
⠸⠇
⠸⠇3 packages are looking for funding
⠸⠇  run `npm fund` for details
⠸⠇
```

```
!npm install -g localtunnel
```

```
⠙⠹ ⠸⠼⠴⠦⠧⠇⠏⠋⠙⠹ ⠸⠼⠴⠦⠧⠇⠏⠋⠙⠹
changed 22 packages in 2s
⠙⠹
⠙⠹3 packages are looking for funding
⠙⠹  run `npm fund` for details
⠙⠹
```

```
!pip install streamlit
```

```
Requirement already satisfied: streamlit in /usr/local/lib/python3.12/dist-packages (1.52.2)
Requirement already satisfied: altair!=5.4.0,!=5.4.1,<7,>=4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (5.5.0
Requirement already satisfied: blinker<2,>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (1.9.0)
Requirement already satisfied: cachetools<7,>=4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (6.2.4)
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (8.3.1)
Requirement already satisfied: numpy<3,>=1.23 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.0.2)
Requirement already satisfied: packaging>=20 in /usr/local/lib/python3.12/dist-packages (from streamlit) (25.0)
Requirement already satisfied: pandas<3,>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.2.2)
Requirement already satisfied: pillow<13,>=7.1.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (11.3.0)
Requirement already satisfied: protobuf<7,>=3.20 in /usr/local/lib/python3.12/dist-packages (from streamlit) (5.29.5)
Requirement already satisfied: pyarrow>=7.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (18.1.0)
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.32.4)
Requirement already satisfied: tenacity<10,>=8.1.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (9.1.2)
Requirement already satisfied: toml<2,>=0.10.1 in /usr/local/lib/python3.12/dist-packages (from streamlit) (0.10.2)
Requirement already satisfied: typing-extensions<5,>=4.4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (4.15.0)
```

```
Requirement already satisfied: watchdog<7,>=2.1.5 in /usr/local/lib/python3.12/dist-packages (from streamlit) (6.0.0)
Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in /usr/local/lib/python3.12/dist-packages (from streamlit) (3.1.45)
Requirement already satisfied: pydeck<1,>=0.8.0b4 in /usr/local/lib/python3.12/dist-packages (from streamlit) (0.9.1)
Requirement already satisfied: tornado!=6.5.0,<7,>=6.0.3 in /usr/local/lib/python3.12/dist-packages (from streamlit) (6.5.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.0->streamlit
Requirement already satisfied: jsonschema>=3.0 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.0->
Requirement already satisfied: narwhals>=1.14.2 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.0-
Requirement already satisfied: gitdb<5,>=4.0.1 in /usr/local/lib/python3.12/dist-packages (from gitpython!=3.1.19,<4,>=3.0.7->st
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0->streaml
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0->streamlit) (2025.
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0->streamlit) (202
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->stre
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit) (3.11
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit)
Requirement already satisfied: smmap<6,>=3.0.1 in /usr/local/lib/python3.12/dist-packages (from gitdb<5,>=4.0.1->gitpython!=3.1.
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->altair!=5.4.0,!=5.4.1,<7
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!=5.4.0,!=5
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonschema>
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!=5.4
Requirement already satisfied: rpds-py>=0.7.1 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!=5.4.0,!=
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas<3,>=1.4.
```

```
!apt install nodejs npm -y
!npm install -g localtunnel
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  gyp javascript-common libc-ares2 libjs-events libjs-highlight.js
  libjs-inherits libjs-is-typedarray libjs-psl libjs-source-map
  libjs-sprintf-js libjs-typedarray-to-buffer libnode-dev libnode72
  libnotify-bin libnotify4 libuv1-dev node-abab node-abbrev node-agent-base
  node-ansi-regex node-ansi-styles node-ansistyles node-aproba node-archy
  node-are-we-there-yet node-argparse node-arrify node-asap node-asynckit
  node-balanced-match node-brace-expansion node-builtins node-cacache
  node-chalk node-chownr node-clean-yaml-object node-cli-table node-clone
  node-color-convert node-color-name node-colors node-columnify
  node-combined-stream node-commander node-console-control-strings
  node-copy-concurrently node-core-util-is node-coveralls node-cssom
  node-cssstyle node-debug node-decompress-response node-defaults
  node-delayed-stream node-delegates node-depd node-diff node-encoding
  node-end-of-stream node-err-code node-escape-string-regexp node-esprima
  node-events node-fancy-log node-fetch node-foreground-child node-form-data
  node-fs-write-stream-atomic node-fs.realpath node-function-bind node-gauge
  node-get-stream node-glob node-got node-graceful-fs node-growl node-gyp
  node-has-flag node-has-unicode node-hosted-git-info node-https-proxy-agent
  node-iconv-lite node-iferr node-imurmurhash node-indent-string node-inflight
  node-inherits node-ini node-ip node-ip-regex node-is-buffer
  node-is-plain-obj node-is-typedarray node-isarray node-isexe node-js-yaml
  node-jsdom node-json-buffer node-json-parse-better-errors node-jsonparse
  node-kind-of node-lcov-parse node-lodash-packages node-log-driver
  node-lowercase-keys node-lru-cache node-mime node-mime-types
  node-mimic-response node-minimatch node-minimist node-minipass node-mkdirp
  node-move-concurrently node-ms node-mute-stream node-negotiator node-nopt
  node-normalize-package-data node-npm-bundled node-npm-package-arg
  node-npmlog node-object-assign node-once node-opener node-osenv
  node-p-cancelable node-p-map node-path-is-absolute node-process-nextick-args
  node-promise-inflight node-promise-retry node-promzard node-psl node-pump
  node-punycode node-quick-lru node-read node-read-package-json
  node-readable-stream node-resolve node-retry node-rimraf node-run-queue
  node-safe-buffer node-semver node-set-blocking node-signal-exit node-slash
  node-slice-ansi node-source-map node-source-map-support node-spdx-correct
  node-spdx-exceptions node-spdx-expression-parse node-spdx-license-ids
  node-sprintf-js node-ssri node-stack-utils node-stealthy-require
  node-string-decoder node-string-width node-strip-ansi node-supports-color
  node-tap node-tap-mocha-reporter node-tap-parser node-tar node-text-table
  node-time-stamp node-tmatch node-tough-cookie node-typedarray-to-buffer
  node-unique-filename node-universalify node-util-deprecate
  node-validate-npm-package-license node-validate-npm-package-name
  node-wcwidth.js node-webidl-conversions node-whatwg-fetch node-which
  node-wide-align node-wrappy node-write-file-atomic node-ws node-yallist
  nodejs-doc
Suggested packages:
  apache2 | lighttpd | httpd libjs-angularjs gnome-shell | notification-daemon
  node-nyc
The following NEW packages will be installed:
  gyp javascript-common libc-ares2 libjs-events libjs-highlight.js
  libjs-inherits libjs-is-typedarray libjs-psl libjs-source-map
  libjs-sprintf-js libjs-typedarray-to-buffer libnode-dev libnode72
  libnotify-bin libnotify4 libuv1-dev node-abab node-abbrev node-agent-base
```

```
  node-ansi-regex node-ansi-styles node-ansistyles node-aproba node-archy
```

```
import streamlit as st
from PIL import Image
st.title ("What is your name!")
st.write("My name is Bhumika")
```

```
2026-01-02 09:44:49.974 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-01-02 09:44:49.976 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-01-02 09:44:49.976 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-01-02 09:44:49.977 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-01-02 09:44:49.978 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-01-02 09:44:49.979 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
```

```
!npm install -g localtunnel
!streamlit run app.py --server.headless true&>/content/logs.txt&
!npx localtunnel --port 8501
```

```
⠙⠹ ⠸⠼⠴⠦ ⠧⠇⠏⠋ ⠙⠹⠸⠼⠴⠦ ⠧⠇⠏⠋ ⠙⠹⠸⠼⠴⠦ ⠧⠇⠏⠋ ⠙⠹⠸⠼⠴⠦ ⠧⠇⠏⠋ ⠙⠹⠸⠼
added 22 packages in 5s
⠼
⠼3 packages are looking for funding
⠼  run `npm fund` for details
⠼npm notice
npm notice New major version of npm available! 10.8.2 -> 11.7.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.7.0
npm notice To update run: npm install -g npm@11.7.0
npm notice
⠴⠙⠹ ⠸⠼⠴⠦ ⠧⠇⠏⠋ ⠙⠹
```

	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6	Unnamed: 7	Unnamed
0	Sl. No.	USN	Student Name	LINEAR INTEGRATED CIRCUITS	DISCRETE TIME SIGNAL PROCESSING	APPLICATIONS OF MACHINE LEARNING USING PYTHON	ELECTROMAGNETICS	EMBEDDED SYSTEMS WITH C	DS USIN VERILO
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	Na
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	Na
3	1	1NT23EC064	HARSHAVARDHAN B M	40	23	26	29	NaN	Na
4	2	1NT23EC065	HINDUJA GOWDA R N	48	49	44	40	43	Na
...	...	...	...	...	...	...	...	...	
68	66	1NT24EC418	SATVIK SATEESH NAIK	5	17	9	1	NaN	Na
69	67	1NT24EC415	PUNITH R S	22	23	11	4	17	Na
70	68	1NT24EC416	SAI PRASAD R G	10	9	a	0	NaN	Na
71	69	1NT24EC404	DISHA N	15	31	25	22	NaN	Na
72	70	1NT22EC071	JEEVAN S	NE	NaN	a	15	NaN	Na

73 rows × 12 columns

```
import pandas as pd
rd_att = pd.read_csv('/content/attend - attend.csv')
display(rd_att)
```

	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6	Unnamed: 7	Unnamed
0	Sl. No.	USN	Student Name	LINEAR INTEGRATED CIRCUITS	DISCRETE TIME SIGNAL PROCESSING	APPLICATIONS OF MACHINE LEARNING USING PYTHON	ELECTROMAGNETICS	EMBEDDED SYSTEMS WITH C	DS USIN VERILO
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	Na
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	Na
3	1	1NT23EC064	HARSHAVARDHAN B M	84	85	88	73	NaN	Na
4	2	1NT23EC065	HINDUJA GOWDA R N	79	77	94	82	82	Na
...	...	...	...	...	...	...	...	...	
68	66	1NT24EC418	SATVIK SATEESH NAIK	89	95	87	91	NaN	Na
69	67	1NT24EC415	PUNITH R S	79	95	91	77	NaN	Na
70	68	1NT24EC416	SAI PRASAD R G	84	82	75	77	NaN	Na
71	69	1NT24EC404	DISHA N	79	77	72	91	NaN	Na
72	70	1NT22EC071	JEEVAN S	68	82	84	77	NaN	Na

73 rows × 12 columns

```
import pandas as pd
rd_att = pd.read_csv('/content/attend - attend.csv')
rd_att.info()
display(rd_att)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 73 entries, 0 to 72
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype 
---  ------       --------------  ----- 
 0   Unnamed: 0   71 non-null     object
 1   Unnamed: 1   71 non-null     object
 2   Unnamed: 2   71 non-null     object
 3   Unnamed: 3   71 non-null     object
 4   Unnamed: 4   71 non-null     object
 5   Unnamed: 5   71 non-null     object
 6   Unnamed: 6   71 non-null     object
 7   Unnamed: 7   24 non-null     object
 8   Unnamed: 8   11 non-null     object
 9   Unnamed: 9   16 non-null     object
 10  Unnamed: 10  10 non-null     object
 11  Unnamed: 11  12 non-null     object
dtypes: object(12)
memory usage: 7.0+ KB
```

	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6	Unnamed: 7	Unnamed
0	Sl. No.	USN	Student Name	LINEAR INTEGRATED CIRCUITS	DISCRETE TIME SIGNAL PROCESSING	APPLICATIONS OF MACHINE LEARNING USING PYTHON	ELECTROMAGNETICS	EMBEDDED SYSTEMS WITH C	DS USIN VERILO
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	Na
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	Na
3	1	1NT23EC064	HARSHAVARDHAN B M	84	85	88	73	NaN	Na
4	2	1NT23EC065	HINDUJA GOWDA R N	79	77	94	82	82	Na
...	...	...	...	...	...	...	...	...	
68	66	1NT24EC418	SATVIK SATEESH NAIK	89	95	87	91	NaN	Na
69	67	1NT24EC415	PUNITH R S	79	95	91	77	NaN	Na
70	68	1NT24EC416	SAI PRASAD R G	84	82	75	77	NaN	Na
71	69	1NT24EC404	DISHA N	79	77	72	91	NaN	Na

```
import pandas as pd
rd_att = pd.read_csv('/content/attend - attend.csv')
rd_att.describe()
display(rd_att)
```

	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4	Unnamed: 5	Unnamed: 6	Unnamed: 7	Unnamed
0	Sl. No.	USN	Student Name	LINEAR INTEGRATED CIRCUITS	DISCRETE TIME SIGNAL PROCESSING	APPLICATIONS OF MACHINE LEARNING USING PYTHON	ELECTROMAGNETICS	EMBEDDED SYSTEMS WITH C	DS USIN VERILO
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	Na
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	Na
3	1	1NT23EC064	HARSHAVARDHAN B M	84	85	88	73	NaN	Na
4	2	1NT23EC065	HINDUJA GOWDA R N	79	77	94	82	82	Na
...	...	...	...	...	...	...	...	...	
68	66	1NT24EC418	SATVIK SATEESH	89	95	87	91	NaN	Na